

Python Intermediário

Automatização com Funções e Organização de Dados



Mini-curso | Módulo 2

Funções: A Fábrica de Código

Uma **Função** é um bloco de código reutilizável que executa uma tarefa específica. Evita que você escreva o mesmo código várias vezes.

```
def saudar(nome): return f"Olá, {nome}!" print(saudar("Maria"))
```

Pense nela como uma **máquina modular**: você insere algo (input) e recebe um resultado (output).



Anatomia de uma Função



1. Definição (def)

É o início de tudo. Você escolhe um nome descritivo para sua "máquina".



2. Parâmetros

São as entradas da função. As matérias-primas que ela vai processar.



3. Corpo / Lógica

Onde o trabalho real acontece. Instruções indentadas dentro do bloco.



4. Retorno (return)

O produto final. O valor que a função devolve para quem a chamou.



Estruturas de Dados

Onde e como guardamos as informações do mundo real.

| Listas: Prateleiras Dinâmicas

Uma **Lista** é uma coleção ordenada de itens que podem ser alterados (mutáveis). Use quando a ordem importa.

```
compras = ["Maçã", "Pão", "Leite"]  
compras[0] = "Banana"  
print(compras) # ["Banana", "Pão",
```

Imagine uma **prateleira de inventário**: você pode adicionar, remover ou trocar itens de lugar facilmente.

| Manipulando Listas

-  **append(item)**: Adiciona um item ao final da lista.
-  **remove(item)**: Procura e remove o primeiro item encontrado com aquele valor.
-  **pop(índice)**: Remove e retorna o item de uma posição específica.
-  **sort()**: Organiza os itens (alfabeticamente ou numericamente).
- **# len(lista)**: Retorna o número total de itens na lista.

"Dica: Em Python, a contagem sempre começa no índice 0!"

Tuplas: O Cofre Imutável

☰ Listas []

Mutáveis: Podem ser alteradas livremente.

Uso: Listas de tarefas, inventários, carrinhos de compras.

🔒 Tuplas ()

Imutáveis: Uma vez criadas, não podem ser mudadas.

Uso: Coordenadas GPS, datas de nascimento, configurações fixas.

```
cord = (10, 20) cord[0] = 5 # ERRO! Tuplas protegem seus dados.
```


[illegible]

```
user = { "nome": "Alice", "idade": 30 }
```

```
user = { "nome": "Alice", "idade": 30 }
```

[illegible]

| Sets: Unicidade Garantida

O

DUPLICATAS

O Filtro Perfeito

Os **Sets** (Conjuntos) ignoram itens repetidos automaticamente. Use-os para garantir que cada informação na coleção seja única.

```
numeros = {1, 2, 2, 3, 3} print(numeros) # {1, 2, 3}
```


Qual Estrutura Escolher?

ESTRUTURA	SÍMBOLO	PRINCIPAL CARACTERÍSTICA
Lista	[]	Ordenada e Alterável (Mutável)
Tupla	()	Ordenada e Fixa (Imutável)
Dicionário	{ : }	Mapeamento de Chave e Valor
Set	{ }	Coleção de Itens Únicos e Sem Ordem

Desafio: Gestor de Equipe

Crie uma função chamada `adicionar_membro` que receba uma lista de nomes e um novo nome.

```
def adicionar_membro(equipe, novo_nome): if novo_nome not in equipe: equipe.append(novo_nome)
```

Dica: Use listas para a equipe e funções para a lógica!

Dúvidas?

Parabéns! Você completou o módulo intermediário.



@SeuCurso



ajuda@python.com

Image Sources



https://miro.medium.com/1*t38rVOS_e_Md8RX5VTwErQ.png

Source: medium.com



<https://acctivate.com/wp-content/uploads/2024/07/organize-warehouse-space-systems.png>

Source: acctivate.com



<https://cdn.vertex42.com/ExcelTemplates/Images/contact-list-template.gif>

Source: www.vertex42.com
